

Stacks and Queues

Introduction to Linear Data Structures

Just like Linked Lists and Arrays, **Stacks** and **Queues** are *linear data structures*.

This means their elements are arranged **one after another**, and traversal happens in a specific order.

Both data structures help in managing data efficiently depending on the order in which you want to **add** and **remove** elements.

Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

This means:

- The **last** element that you insert will be the **first** one to be removed.
- You can only insert or remove elements from **one end** — the *top* of the stack.

Think of it like a **stack of plates** in a cafeteria:

- You add new plates on top.
- You remove plates only from the top.
- You can't remove a plate from the middle without disturbing the ones above it.

Structure of a Stack

A Stack supports these main operations:

Operation	Description
push()	Adds an element to the top of the stack
pop()	Removes the top element from the stack
peek() / top()	Returns the top element without removing it
isEmpty()	Checks if the stack is empty
display()	Shows all elements in the stack

Stack Implementation Using Python (List)

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, data):
        self.stack.append(data)
        print(f"{data} pushed to stack")

    def pop(self):
        if self.is_empty():
            print("Stack is empty! Nothing to pop.")
            return
        removed = self.stack.pop()
        print(f"{removed} popped from stack")

    def peek(self):
        if self.is_empty():
            print("Stack is empty! No top element.")
            return
        print(f"Top element is: {self.stack[-1]}")

    def is_empty(self):
        return len(self.stack) == 0

    def display(self):
        print("Stack (top to bottom):")
        for item in reversed(self.stack):
            print(item)
```

Step-by-Step Explanation

1. `__init__` → Initializes an empty list to represent the stack.
2. `push()` → Adds a new element to the top using `.append()`.
3. `pop()` → Removes the last added element using `.pop()`. Check if the stack is empty first.
4. `peek()` → Accesses the last element (`self.stack[-1]`) without removing it.
5. `is_empty()` → Returns `True` if stack is empty, else `False`.

6. `display()` → Prints stack elements from top to bottom.

Time and Space Complexity

Operation	Time Complexity	Space Complexity
Push	$O(1)$	$O(n)$
Pop	$O(1)$	$O(n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$

Real-Life Examples of Stack

- Undo/Redo in text editors
- Browser back button (last page visited is first to go back)
- Function call stack in programming
- Reversing strings

Queue

A **Queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle.

This means:

- The **first** element added to the queue will be the **first** one to be removed.
- Insertion happens at one end called the **rear**, and deletion happens at the other end called the **front**.

Think of it like a **line of people waiting for a bus**:

- The first person to join the line is the first one to board the bus.
- New people join at the end of the line.

Structure of a Queue

Operation	Description
enqueue()	Adds an element at the rear end
dequeue()	Removes an element from the front end
peek() / front()	Returns the element at the front
isEmpty()	Checks if the queue is empty
display()	Shows all elements in the queue

Queue Implementation Using Python (List)

```
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, data):
        self.queue.append(data)
        print(f"{data} added to queue")

    def dequeue(self):
        if self.is_empty():
            print("Queue is empty! Nothing to remove.")
            return
        removed = self.queue.pop(0)
        print(f"{removed} removed from queue")

    def peek(self):
        if self.is_empty():
            print("Queue is empty! No front element.")
            return
        print(f"Front element is: {self.queue[0]}")

    def is_empty(self):
        return len(self.queue) == 0

    def display(self):
        print("Queue (front to rear):")
        for item in self.queue:
            print(item)
```

Step-by-Step Explanation

1. `__init__` → Initializes an empty list to represent the queue.
2. `enqueue()` → Adds new data to the rear using `.append()`.
3. `dequeue()` → Removes the first element using `.pop(0)`.
4. `peek()` → Accesses the first element (`self.queue[0]`) without removing it.
5. `is_empty()` → Checks if the queue is empty.
6. `display()` → Prints queue elements in the order they'll be removed.

Time and Space Complexity

Operation	Time Complexity	Space Complexity
Enqueue	$O(1)$	$O(n)$
Dequeue	$O(n)$ (due to shifting)	$O(n)$
Peek	$O(1)$	$O(1)$

Note: Using `collections.deque` instead of a list makes Enqueue and Dequeue both $O(1)$.

Queue Variants

1. **Circular Queue** – connects rear to front, making efficient use of space.
2. **Priority Queue** – each element has a priority, and the element with the highest priority is served first.
3. **Double-Ended Queue (Deque)** – elements can be added or removed from both ends.

Real-Life Examples of Queue

- Waiting line systems (bank, restaurant, ticket counter)
- CPU scheduling (first process that arrives gets executed first)
- Printing documents (first file sent is printed first)
- Task management systems

Comparison Table: Stack vs Queue

Feature	Stack	Queue
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion	Top	Rear
Deletion	Top	Front
Example	Stack of plates	Line of people
Use cases	Undo/Redo, Function calls	Scheduling, Messaging systems

Summary

- **Stacks** → Best for problems requiring *reverse order* or *backtracking*.
- **Queues** → Best for problems requiring *sequential processing* or *first-come-first-serve* order.
- Both are **linear**, **dynamic**, and **used in algorithms** like BFS, DFS, parsing, and scheduling.